

HACKATHON SQUAD

Maximum Weight Independent Set

Algorithm Design & Submission Report

May 2026

1. The Problem

Our university is throwing its flagship hackathon, and the goal is straightforward: build the strongest possible team from a pool of freshman coders. Each coder comes with a Skill Rating that captures how fast and sharp they are. The complication is that some pairs of coders have old rivalries — past department drama, old feuds — and they simply cannot be on the same team.

So the task is to pick a group of coders that avoids every conflict pair while making the combined Skill Rating as large as it can possibly be. In graph theory, this is called the Maximum Weight Independent Set problem. Every coder is a node, every rivalry is an edge, and we need the heaviest subset of nodes with no edges between them.

□ **Why this is hard:** This problem is NP-Hard, which means no known algorithm can solve it exactly in reasonable time for large inputs. With up to 200,000 coders in the pool, we cannot afford to check every possible subset. Instead, we use smart approximation techniques to find the best team we can within the five-minute time window.

Constraints

Parameter	Limit
Number of coders (N)	1 to 200,000
Conflict pairs (M)	0 to $N(N-1)/2$ — can be as dense as a complete graph
Skill rating per coder	1 to 1,000,000,000
Time limit	5 minutes. We use 285 seconds, leaving a 15-second safety buffer.
Indexing	Coders numbered 1 to N. All conflict pairs are distinct. No self-loops.

2. Our Strategy

We designed a four-phase pipeline that gets progressively smarter over the five-minute window. The first ~20 seconds are spent constructing several strong starting teams using different greedy strategies. After that, we hand control to Simulated Annealing, which runs for the remaining ~265 seconds and keeps searching for better teams right up to the buzzer.

Phase	Name	What It Does	Time Budget
1	Multi-Strategy Greedy	Build four different starting teams using four different node-ordering strategies	~20 seconds
2	Local Search Polish	Squeeze every last point out of each greedy team before handing it to SA	Shared with Phase 1
3	Simulated Annealing	Run millions of ADD / DROP / SWAP moves on the best starting team, guided by a cooling schedule	~265 seconds
4	Random Restart	Every 30 seconds, reload the best solution found so far and shake it up a little so SA can't get stuck	Inside Phase 3

Phase 1 — Building Four Different Starting Teams

A single greedy pass will always miss things, because the order you consider coders in completely changes who ends up on the team. We run four passes in parallel, each with a different sense of priority:

- **Density first** — take whichever coder has the best skill-to-rivalries ratio. High value, low drama.
- **Raw skill first** — always pick the highest-rated available coder, pure and simple.
- **Fewest conflicts first** — start with whoever has the fewest rivalries, keeping the most future options open.
- **Net value first** — skill minus a penalty for the value of people you would be blocking. This one thinks a step ahead.

We keep all four results and carry the best one forward into Simulated Annealing. Starting SA from a strong point means it spends its time finding improvements rather than climbing out of a hole.

Phase 2 — Polishing Each Team

After each greedy build, we immediately run a local search to tighten the result:

- **Fill pass** — scan through every coder who isn't on the team yet. Add anyone with no conflicts against the current roster.
- **Swap pass** — for each current team member, check whether removing them would free up higher-rated coders who were being blocked. If the swap is a net gain, make the trade.

This runs in a loop until no more improvements can be found, or the time slice runs out.

Phase 3 — Simulated Annealing

SA is the heart of the solution. It runs for the bulk of the time and makes millions of small adjustments, guided by a temperature parameter that controls how adventurous the search is allowed to be. It starts hot — willing to accept worse moves to explore broadly — and gradually cools down, becoming more conservative as it homes in on the best solution it has found.

Three types of moves are available at each step:

- **ADD** — if a coder isn't on the team and doesn't conflict with anyone currently on it, always add them. Free skill is free skill.
- **DROP** — occasionally remove someone from the team, even at a skill cost. This lets SA escape local optima — situations where everything locally looks like a dead end but better territory lies just a few moves away. The hotter the temperature, the more willing SA is to take this kind of hit.
- **SWAP** — remove everyone on the team who conflicts with coder u, then add u. Accept the trade if it is profitable, or sometimes accept it even if it is not, based on the current temperature.

Every time SA beats its own record, we immediately run a polish pass on the new best team to lock in any easy additions. SA then reloads from that polished state and keeps going.

Phase 4 — Shaking Things Up Every 30 Seconds

Long SA runs have a tendency to settle too deeply into one region of the solution space. Every 30 seconds, we pull the best solution found so far, randomly drop about 10% of the team members, and let SA naturally refill those gaps. This forces exploration into neighbourhoods the cooling schedule alone would never visit.

Adaptive Heat Boost

If 500,000 SA iterations pass without any progress, we double a heat multiplier that sits on top of the cooling formula. This temporarily raises the effective temperature so the search can make wilder moves and escape deep ruts. The boost fades gradually — 5% per clock check — so it doesn't undo all the cooling. It is a targeted nudge, not a full restart.

3. Input and Output

Input Format

The first line gives N and M — how many coders there are and how many conflict pairs. The second line lists all N skill ratings separated by spaces. The next M lines each describe one conflict pair.

```
N M
S1 S2 S3 ... SN
u1 v1
u2 v2
...
uM vM
```

Output Format

Two lines. First, the total skill rating of the selected team. Second, the indices of the chosen coders in ascending order, separated by spaces.

```
<total skill rating>
<index1> <index2> ... <indexK>
```

Worked Example

Four coders with skills 5, 3, 4, 6 and three rivalries: coders 1 and 2 clash, 2 and 3 clash, 3 and 4 clash. Coder 4 has the highest individual skill (6) but conflicts with coder 3. If we pick coder 4, we lose coder 3, and the best we can do is {4} with score 6. If instead we pick coders 1 and 3, neither conflicts with the other, and we score 5 + 4 = 9.

Input:	Output:
4 3	9
5 3 4 6	1 3
1 2	
2 3	
3 4	

4. How to Compile and Run

□ **Requirements:** A C++17-compatible compiler — g++ version 7 or later, or clang++ version 5 or later. No external libraries or dependencies. Everything used is part of the C++ Standard Library.

Step 1 — Save the source file

Save the submitted source code to a file called:

```
hackathon_squad_final.cpp
```

Step 2 — Compile

Open a terminal in the same folder and run one of these commands depending on your system:

```
# Linux or macOS
g++ -O2 -std=c++17 -o hackathon_squad hackathon_squad_final.cpp

# Windows (MinGW / MSYS2)
g++ -O2 -std=c++17 -o hackathon_squad.exe hackathon_squad_final.cpp

# Maximum performance — recommended for the actual judge
g++ -O3 -std=c++17 -o hackathon_squad hackathon_squad_final.cpp
```

Step 3 — Run with an input file

Put your test input in a plain text file (e.g. input.txt) and redirect it into the program:

```
# Linux / macOS
./hackathon_squad < input.txt

# Windows
hackathon_squad.exe < input.txt

# Save output to a file at the same time
./hackathon_squad < input.txt > output.txt
```

Step 4 — Type input manually

You can also type the input directly in the terminal if you want to try a quick test case. When you are done entering input, press Ctrl+D on Linux/macOS or Ctrl+Z then Enter on Windows.

```
./hackathon_squad
4 3
5 3 4 6
1 2
2 3
3 4
^D
```

Step 5 — Expected output for the example

```
9
1 3
```

❏ **Note on runtime:** The program is designed to run for up to 285 seconds, continuously finding better teams. On small inputs it will finish in a fraction of a second. On large inputs, let it run — every extra second is another chance to improve the score.

Compiler Flag Reference

Flag	What It Does
-O2 / -O3	Turns on compiler optimisations. Makes the SA loop run several times faster. Always use this on the judge.
-std=c++17	Required for the language features used in the code.
-o <name>	Sets the name of the compiled executable.

5. Complexity at a Glance

Everything in the algorithm is designed to be as lightweight per step as possible, so SA can pack in as many moves as the five-minute window allows. Here is the time cost of each component:

Component	Time per call	Notes
Greedy Build (each ordering)	$O(N + M)$	Run four times total, once per ordering
Local Search Round	$O(N + M)$	Repeated until no improvement or the time slice ends
SA Move — ADD or DROP	$O(\deg(u))$	Only touches the conflict counters of u 's neighbours
SA Move — SWAP	$O(\deg(u))$	Removes in-set neighbours of u , then adds u
Random Restart (every 30s)	$O(N + M)$	Full conflict-count recompute when reloading the best solution

Space complexity is $O(N + M)$ for the adjacency list, plus $O(N)$ for the `inSet`, `conflicts`, and `bestInSet` arrays. Well within limits for $N = 200,000$.

6. Why We Made These Choices

Why Simulated Annealing?

The MWIS solution space is riddled with local optima — positions where every individual change looks like it would hurt, but much better solutions exist just a handful of moves away. Pure greedy and local search reliably get trapped in these spots and cannot escape.

SA's key advantage is that it can accept worsening moves, especially early on when temperature is high. This lets it explore broadly before gradually narrowing in. With five minutes of runtime we can run tens of millions of SA iterations, which makes it genuinely effective rather than just theoretically appealing.

Why Four Greedy Orderings?

Different orderings often find solutions in completely different parts of the search space. Density-first tends to find compact, high-value teams. Degree-first finds large teams by keeping options open. Raw-skill-first sometimes stumbles on a spectacular team the others miss entirely. Running four in parallel and taking the best is cheap insurance — it costs only ~20 seconds — and it means SA starts from the strongest possible position rather than a mediocre one.

Why Remove ALL Conflicting Members in the SWAP Move?

The idea behind SWAP is to trade everyone who conflicts with coder u for coder u alone. That only works if u actually gets added. If we remove just one conflicting member, u almost certainly still has others blocking it, and the whole move gets reverted — it is a lot of work for nothing. Removing every in-set neighbour of u guarantees u 's conflict count drops to zero before we try to add it, making SWAP a genuinely useful move rather than an elaborate no-op.

Why a Heat Multiplier Instead of Setting Temperature Directly?

The SA loop recalculates `curTemp` from the geometric cooling formula every 4,096 iterations. In optimised C++, 4,096 iterations takes well under a millisecond. If we bump `curTemp` directly, the very next clock check overwrites it before a single meaningful move has used the extra heat. The `heatBoost` multiplier sits on top of the cooling formula and decays gradually over several seconds, so the temperature elevation actually has time to influence the search.

7. The Output Is Always a Valid Team

Regardless of which path the algorithm takes, the final answer is guaranteed to be a legitimate conflict-free team. This holds because the algorithm maintains a strict invariant throughout: at every single moment, the `inSet` array represents a valid independent set. Here is how each operation preserves that:

1. The greedy phase only adds coder `u` when their conflict counter is exactly zero — no current team member is their rival.
2. Local search uses the same zero-conflict check before adding anyone.
3. The SA ADD move only fires when `conflicts[u] == 0`.
4. The SA SWAP move removes every in-set rival of `u` before adding `u`, so `u`'s conflict count is guaranteed to be zero at the point of addition.
5. The SA DROP move only removes someone — it can never introduce a new conflict.
6. `bestInSet` is only updated when a new best score is reached from an already-valid SA working state.

✓ **Core invariant:** The `conflicts[]` array is updated atomically alongside every add and remove operation. It never drifts out of sync with `inSet`. This means we never accidentally output a team where two members have a rivalry.